

## CONSOLIDATED PRODUCT BRIEF

# Private AI Workspace

---

## Local multimodal models + Knowledge + Memory + Skills + native tools + MCP

A synthesis of the product concept, architecture, MVP, monetization, competitive positioning, technical decisions, and unresolved questions discussed in this conversation.

Working name discussed: Nook (candidate only; not validated).

### Core promise:

A private AI workspace that runs on the user's iPhone, knows the information they choose to give it, remembers useful context across conversations, and can act through explicitly granted local and external tools.

### The product in one line

Know. Remember. Act. - powered by a downloadable local LLM/VLM, with clear boundaries whenever data leaves the device.

### Why this is not just another local LLM runner

- Knowledge: chat with personal documents without sending them to a cloud LLM.
- Memory: retrieve prior conversations, people, places, topics and decisions with provenance.
- Abilities: install Skills that teach workflows without granting arbitrary executable code.
- Actions: native iOS tools for a small set of device capabilities, plus MCP for open-ended external integrations.
- Multimodality: understand text, images, screenshots and document pages.
- Privacy: local inference and local retrieval by default; explicit approval for external MCP calls.

# 1. Product thesis and positioning

## Target product

An iOS-first personal AI workspace, not a generic chatbot and not primarily a model playground. Users download a capable local model, add Knowledge, optionally enable Memory, install Skills, and connect external services through MCP.

## Consumer-facing mental model

| Pillar       | What users see                                   | Underlying technology                                                         |
|--------------|--------------------------------------------------|-------------------------------------------------------------------------------|
| Know         | My Knowledge: PDFs, notes, images, project files | Local extraction, chunking, embeddings, hybrid retrieval/RAG                  |
| Remember     | Recall prior chats, people, places, decisions    | Conversation index, summaries, entities, mentions, semantic/lexical retrieval |
| Act          | Skills and connected tools                       | Skill instructions, ToolRegistry, native Swift tools, MCP                     |
| See          | Understand photos, screenshots, PDF pages        | Local VLM / multimodal prompting                                              |
| Stay private | On-device by default                             | Local inference, local DB/vector index, explicit external-call boundary       |

## Positioning

Recommended pitch: “A private AI workspace that knows your information and connects to your tools.”

Potential tagline: “Your AI. On your device. On your terms.”

Alternative product sentence: “Give your AI knowledge and abilities without giving away your data.”

## What not to lead with

- GGUF, MLX, quantization, embeddings, HNSW, RAG, context windows or function schemas.
- “AI that controls your iPhone” - iOS native capabilities are intentionally bounded.
- Model-selection complexity. Advanced controls should exist, but not define onboarding.

CLOSED - The differentiation is the integrated system - local AI + Knowledge + Memory + Skills + tools - not local inference alone.

## 2. High-level architecture

The architecture should keep intelligence, behavior, capabilities, retrieval and context assembly separate.

SwiftUI App → AgentSession → SkillManager / ToolRegistry / ContextAssembler → Local Model Runtime.  
ToolRegistry routes Native, MCP and RAG/Memory tools. Knowledge and Memory remain local retrieval systems.

### Core architectural boundaries

| Component          | Responsibility                                                                              |
|--------------------|---------------------------------------------------------------------------------------------|
| ModelRuntime       | Load/download local LLM/VLM, tokenize, stream generation, structured/tool output.           |
| AgentSession       | Own the inference/tool loop and conversation state.                                         |
| SkillManager       | Discover, install, activate and permission Skills; progressive disclosure of SKILL.md.      |
| ToolRegistry       | Expose one normalized tool interface regardless of implementation.                          |
| ContextAssembler   | Enforce token budgets across system prompt, skill, recent chat, retrieval and tool results. |
| KnowledgeEngine    | Document ingestion, embeddings, retrieval and citations.                                    |
| MemoryEngine       | Cross-chat retrieval, summaries, entities, mentions and provenance.                         |
| MCPClient          | Connect to remote Streamable HTTP servers, discover and execute tools.                      |
| NativeToolExecutor | Bridge selected capabilities to EventKit, Core Location, Files, etc.                        |

### Unified tool abstraction

The model should see semantic tools such as `calendar.create_event`, `github.search_issues`, `documents.search` and `memory.search`. It should not need to know whether execution is native Swift, MCP or a local retrieval engine.

**CLOSED** - Use a unified ToolRegistry; do not leak Gallery-style `run_intent/run_js/run_mcp` routing into the model-facing interface.

### 3. Local model runtime and model strategy

#### Preferred initial runtime

MLX Swift is the preferred MVP direction because the product is iOS-first and multimodality is core. MLX requires MLX-compatible model weights; arbitrary GGUF cannot be loaded directly.

#### Why MLX first

- Apple-Silicon-focused runtime and strong Swift integration.
- Good fit for local LLM and VLM workloads.
- Current ecosystem includes Qwen, Gemma, Llama, Phi and other families.
- Structured generation/tool calling can be layered into the agent runtime.
- Avoids dual-runtime complexity during MVP.

#### Why llama.cpp remains strategically useful

- Much broader GGUF ecosystem and easy arbitrary-model import.
- Strong Metal support and cross-platform portability.
- Useful for advanced/power-user model compatibility later.
- Can benchmark the same logical model against MLX to choose per-device/runtime strategies.

OPEN - MLX-only long term vs MLX for curated models + llama.cpp/GGUF for advanced imports. MVP direction: MLX first.

#### Candidate model profile

| Model         | Approx tier    | Vision | MLX              | llama.cpp                      | Likely role                                   |
|---------------|----------------|--------|------------------|--------------------------------|-----------------------------------------------|
| Qwen 3.5 4B   | Balanced       | Yes    | Strong candidate | Validate exact VLM path        | Potential flagship                            |
| Qwen3-VL 4B   | Balanced       | Yes    | Yes              | Validate exact current support | Multimodal flagship                           |
| Gemma 3 4B    | Balanced       | Yes    | Yes              | Yes                            | Dual-runtime benchmark / alternative flagship |
| Qwen2.5-VL 3B | Efficient      | Yes    | Yes              | Yes                            | Older-device VLM baseline                     |
| SmolVLM2 ~2B  | Fast           | Yes    | Yes              | Yes                            | Low-memory multimodal                         |
| Qwen3 4B      | Balanced text  | No     | Yes              | Yes                            | Agent/text specialist                         |
| Llama 3.2 3B  | Balanced text  | No     | Yes              | Yes                            | Ecosystem compatibility                       |
| Phi-3.5 Mini  | Balanced text  | No     | Yes              | Yes                            | Structured/reasoning benchmark                |
| Gemma 3 1B    | Fast           | No     | Yes              | Yes                            | Utility / low-memory model                    |
| SmolLM3 3B    | Efficient text | No     | Yes              | Yes                            | Open text/RAG baseline                        |

## Benchmark that should decide the flagship

- Time to first token, decode tokens/sec, peak RAM, energy and thermal behavior on actual iPhones.
- RAG grounded QA and citation correctness.
- Tool selection, JSON validity, parameter correctness and unnecessary tool-call rate.
- Skill-instruction adherence.
- Vision extraction/reasoning on screenshots, receipts, charts, invitations and PDF pages.
- Multi-step agent loops: Skill → RAG → MCP/native tool → final answer.

CLOSED - Choose models based on the app workload, not generic benchmark scores.

## 4. Skills: behavior, not executable privilege

### Definition

A Skill is a portable package of instructions and metadata that teaches the agent how to perform a class of tasks using capabilities that already exist.

Minimal portable form: a folder containing SKILL.md. An optional app-specific manifest can request tools, Knowledge scopes and metadata.

### Progressive disclosure

Initially inject only skill name + description. When the agent selects a relevant Skill, load the full SKILL.md. This keeps context small even with many installed Skills.

### Permissions

A Skill may request tools, but installation never grants them automatically. The user grants or denies tool and Knowledge access per Skill.

| User action                              | Allowed?                   | Notes                                        |
|------------------------------------------|----------------------------|----------------------------------------------|
| Install/import a Skill                   | Yes                        | Recommended free and unlimited.              |
| Inspect SKILL.md                         | Yes                        | Important for trust.                         |
| Enable/disable Skill                     | Yes                        | Per installation.                            |
| Grant Knowledge scopes                   | Yes                        | Explicitly controlled.                       |
| Grant native/MCP tools                   | Yes                        | Only tools already registered/connected.     |
| Invent a new executable tool in Markdown | No                         | Tool must exist in ToolRegistry.             |
| Modify imported Skill                    | Prefer duplicate/customize | Preserves upstream identity and update path. |
| Create personal Skill                    | Yes                        | Limited free; unlimited Pro was discussed.   |

### JavaScript Skills

Do not include JavaScript execution in MVP. If later required, add a separately permissioned sandboxed ScriptTool executor for deterministic transformations, parsers, calculations or CSV/JSON manipulation. Privileged iOS capabilities remain native Swift tools.

CLOSED - Skills and Tools are separate concepts.

CLOSED - No JavaScript Skills in MVP.

CLOSED - Imported Skills should not automatically gain requested capabilities.

## 5. Native iOS capabilities and MCP

### Native tools

Native iOS capabilities are limited by Apple frameworks, sandboxing, permissions, entitlements and App Store policy. They should be useful primitives, not the core product promise.

#### MVP candidates

- Calendar: search/read and create events.
- Reminders: search/read and create.
- Current date/time.
- Current location.
- Files/photo selection and camera/image input.
- Possibly local notification scheduling.
- Share Extension to accept content from Safari, Photos, Files and other apps.

### MCP

MCP is the extensibility mechanism for external services. Capability-specific business logic runs on the MCP server; the app only needs a generic MCP client and agent-tool adapter.

#### MVP MCP flow

1. Add a remote MCP server URL.
2. Support Streamable HTTP and custom headers/auth.
3. Discover server tools and input schemas.
4. Normalize discovered tools into AgentTool entries.
5. Let users enable/disable tools.
6. Ask for approval before external/consequential calls.
7. Execute remotely and return tool result to AgentSession.
8. Continue local model inference with the result.

### Privacy boundary

The UI should distinguish on-device execution from external tool calls. A local/private positioning requires explicit disclosure of the exact content being sent to an MCP server when practical.

**CLOSED** - Remote Streamable HTTP MCP first.

**CLOSED** - External calls require clear user control/approval policy.

## 6. Knowledge: local RAG without bloating context

### User experience

Call the feature Knowledge, not RAG. Users create collections such as Personal, Project Alpha or University and add PDFs, notes, Markdown, images and other supported files. Chats can scope retrieval to selected collections.

### Ingestion pipeline

Document → text/structure extraction → semantic chunking → local embedding model → chunk metadata + embedding → local index.

### Chunk record

- documentID
- text
- page/section
- parent section reference
- embedding reference
- source offsets when useful for citations

### Chunking direction

Start around 400-600 tokens with modest 50-100 token overlap, but preserve headings/paragraph boundaries ahead of exact size. Parent/child chunking can be added later.

### Retrieval

Use hybrid retrieval where practical: semantic vectors + lexical/BM25/FTS. Retrieve perhaps 15-30 candidates, deduplicate/merge, then select roughly 4-6 chunks under an explicit token budget. A dedicated reranker can come later.

### RAG as tools

- `documents.search(query, collection?, limit?)`
- `documents.get_chunk(id)`
- `documents.get_page(document, page)`
- `documents.list()`

### Multimodal RAG

Eventually store both extracted text and rendered page images. Text can locate a relevant PDF page; the VLM can then inspect the page image when charts, tables or diagrams matter.

**CLOSED** - Never place whole documents in the model context; retrieve only relevant evidence.

**CLOSED** - Use a separate small local embedding model rather than the generative chat model.

**OPEN** - Exact embedding model and embedding dimension.

**OPEN** - Multimodal PDF-page retrieval in MVP vs post-MVP. Current direction: post-MVP unless cheap to add.



## 7. Context management

Context size is a product constraint. ContextAssembler should own all inclusion decisions instead of allowing Skills, RAG, chat history or tool results to append content independently.

### Illustrative 8K budget

| Context source            | Illustrative budget |
|---------------------------|---------------------|
| System instructions       | ~800-1,000 tokens   |
| Active Skill              | ~500-1,000          |
| Recent chat / summary     | ~1,500-2,000        |
| Knowledge/Memory evidence | ~2,000-3,000        |
| Tool results              | ~500-1,000          |
| Output reserve            | ~1,500-2,000        |

Budgets should be dynamic by model context size and task, but the principle is fixed: reserve output space and cap every contributor.

CLOSED - ContextAssembler is first-class infrastructure, not a prompt-building helper scattered across features.

## 8. Long-term Memory and cross-conversation recall

### Desired user experience

Users should eventually be able to ask “What was that restaurant I mentioned last week?”, “What database did Sarah recommend in London?”, or explicitly attach a previous conversation as context.

### Three product concepts

| Concept      | Meaning                                                                                                             |
|--------------|---------------------------------------------------------------------------------------------------------------------|
| Chat History | Raw conversation record; searchable/openable.                                                                       |
| Memory       | Automatically retrievable context from prior interactions: topics, decisions, people, places and relevant messages. |
| Knowledge    | Files and material deliberately added by the user.                                                                  |

### MVP memory direction

- Persist all conversations locally.
- Create conversation summaries.
- Chunk/embed prior conversations for semantic retrieval.
- Add lexical/FTS search.
- Allow explicit “Previous conversation” attachment.
- Return provenance links to exact original messages.

### Later structured memory

- Entity extraction: person, place, organization, topic/thing.
- Entity aliases and entity resolution.
- Entity mentions linked to exact source message.
- Optional relationships such as person → recommended → technology.
- User review/forget controls and reprocessing of derived memory.

OPEN - Structured entity graph in MVP. Current direction: semantic/lexical cross-chat retrieval first; entity graph after validation.

## 9. Chat, entity and memory storage

### Recommended source-of-truth database

SQLite + GRDB is the preferred relational store for conversations, messages, entities, mentions, relations, summaries and FTS5. A separate vector index can be used for semantic retrieval.

### Core relational model

| Table               | Key fields / purpose                                                    |
|---------------------|-------------------------------------------------------------------------|
| Conversation        | id, title, timestamps, summary                                          |
| Message             | id, conversationId, role, content, createdAt                            |
| Entity              | id, type, canonicalName, normalizedName, summary                        |
| EntityAlias         | entityId, alias, normalizedAlias                                        |
| EntityMention       | entityId, messageId, surfaceText, offsets, confidence, extractorVersion |
| EntityRelation      | subjectEntityId, predicate, objectEntityId, sourceMessageId, confidence |
| MemoryChunk         | conversationId, message range, text, embeddingId                        |
| ConversationSummary | conversationId, summary, embeddingId                                    |

### Provenance rule

Every derived memory should point back to evidence in the raw conversation. Raw messages are source data; entities, relations, summaries and embeddings are rebuildable indexes.

**CLOSED** - Use relational storage rather than introducing a graph database.

**CLOSED** - Raw chat is source-of-truth; derived memory can be deleted/rebuilt as extraction improves.

## 10. Vector storage and search

### Options discussed

| Option                       | Pros                                                      | Cons / status                                               |
|------------------------------|-----------------------------------------------------------|-------------------------------------------------------------|
| ObjectBox                    | Native Swift; on-device HNSW; metadata + vectors together | Additional dependency; benchmark before commitment          |
| SQLite + brute-force vectors | Simple architecture; may be enough for 1K-20K chunks      | Linear scan; may need optimization/Accelerate               |
| SQLite + vector extension    | Single DB possibility                                     | iOS packaging/maturity needs validation                     |
| LanceDB                      | Embedded vector-oriented design                           | Swift/iOS integration less mature / more binding complexity |

### Current recommendation

Use SQLite/GRDB for relational truth and FTS5. Benchmark brute-force vector similarity against ObjectBox HNSW on representative iPhones and realistic corpus sizes before choosing the vector layer.

OPEN - ObjectBox vs brute-force/local vector index. This should be decided by measurement, not preference.

## 11. MVP scope

The MVP should prove local intelligence, personal Knowledge and extensibility without turning into a model-runtime laboratory.

### A. Local AI

- Download 2-3 curated local models.
- Streaming local inference.
- Local chat history.
- Model switching and basic storage management.
- Advanced model controls hidden behind an advanced screen.

### B. Multimodal

- Attach images/screenshots.
- Camera input.
- File attachment.
- At least one strong local VLM.

### C. Knowledge

- PDF/TXT/Markdown import; DOCX if extraction cost is acceptable.
- Knowledge collections.
- Local extraction/chunking/embeddings.
- Basic semantic retrieval; lexical/FTS if practical.
- Source/page citations.
- Per-chat Knowledge scope.

### D. Skills

- Several built-in Skills.
- Import/install SKILL.md packages.
- Enable/disable and inspect instructions.
- Progressive Skill loading.
- Per-Skill tool/Knowledge permissions.
- Personal Skill creation/editing in a limited free tier.

### E. MCP

- Remote Streamable HTTP.
- Tool discovery.
- Custom headers/auth.
- Enable/disable tools.
- Approval before external/consequential calls.
- Tool result integrated into the same agent loop.

### F. Native / iOS integration

- Calendar read/create.
- Reminders read/create.

- Current time and location.
- File/photo selection.
- Share Extension.

### Explicitly defer

- JavaScript Skills.
- Skill marketplace.
- Voice/background assistant.
- Cloud model providers.
- Multi-agent systems.
- Fine-tuning/training.
- Complex cloud sync.
- Large set of native integrations.
- Full structured entity graph if MVP retrieval works without it.

## 12. MVP demonstration / acceptance scenario

The strongest demo should show the integrated value in under a minute.

9. User downloads the recommended local VLM.
10. User creates “Project Alpha” Knowledge and imports project PDFs.
11. User asks: “What are the biggest risks in this project?” The app answers from local retrieval with page citations.
12. User attaches a screenshot and asks: “Does this implementation match the specification?” The VLM combines the screenshot with retrieved project evidence.
13. User asks: “Check GitHub for open issues related to these problems.” The app shows the external-data approval boundary, invokes a GitHub MCP tool, and combines the result with local context.

This demonstrates local inference + Knowledge + multimodality + Skills/agent behavior + MCP + privacy boundaries without explaining the underlying jargon.

# 13. Product UX and information architecture

## Primary areas

| Area        | Purpose                                                                             |
|-------------|-------------------------------------------------------------------------------------|
| Chat        | Normal conversation, attachments, tools and scoped Knowledge.                       |
| Knowledge   | Collections, documents, indexing status and citations.                              |
| Skills      | Built-in, imported and personal Skills; permissions and configuration.              |
| Connections | MCP servers and available tools.                                                    |
| Memory      | Eventually: searchable prior context, people/places/topics, review/forget controls. |
| Models      | Simple curated choices; advanced model controls behind a secondary screen.          |

## Share Sheet

Share → app should support Ask AI, Add to Knowledge, Summarize and Open in Chat. This is an important iOS-native way to receive content that the app cannot otherwise inspect due to sandboxing.

## Model onboarding

Use human tiers such as Fast, Balanced (Recommended) and Powerful rather than exposing quantization names in first-run UX.



## 14. Monetization strategy discussed

### Principle

Do not meter local messages/tokens. The user supplies the compute, so artificial token quotas would conflict with the local-first value proposition.

### Free should prove the product

- Unlimited local chat.
- At least one genuinely capable local model; ideally 2-3 curated options.
- Basic multimodality.
- Chat history.
- Basic Knowledge/RAG with a reasonable free limit.
- Built-in Skills.
- Unlimited import/use of community Skills.
- Limited personal Skill creation.
- One MCP server.
- Basic native tools.

### Pro should make it deeply personal

- Long-term Memory / cross-conversation recall.
- Unlimited Knowledge collections/documents.
- Advanced retrieval and multimodal Knowledge.
- Unlimited personal Skills and advanced Skill configuration.
- Unlimited MCP servers and richer saved permissions.
- Advanced model import/settings.
- Advanced export/backup/privacy controls.

### Pricing direction

A one-time/lifetime Pro purchase was favored initially because local inference has little marginal server cost. A subscription becomes more defensible only when recurring-cost services exist, such as encrypted sync, hosted gateways, backups or marketplace infrastructure.

### Skill monetization nuance

Importing community Skills should be free and unlimited. Skill consumption can become an acquisition channel. Monetize deeper personalization/authoring rather than ecosystem access.

CLOSED - Do not put a message/token meter on local inference.

CLOSED - Imported community Skills should be free to install/use.

OPEN - Exact Free/Pro limits and price. Discussed examples are hypotheses, not decisions.

## 15. Competitive takeaway

Existing iOS apps already cover many individual pieces: local GGUF/MLX inference, multimodal chat, document RAG and increasingly MCP. Therefore “local LLM + documents + MCP” is not enough differentiation by itself.

### Where the whitespace appears stronger

The differentiated system is a coherent private personal AI centered on Knowledge + long-term Memory + installable Skills + explicit capabilities, with local multimodal models underneath.

Structured, trustworthy Memory with provenance may become more defensible than adding yet another model format because it compounds in value over months of use.

### Competitive framing

| Category             | Strength                                                 | Gap relative to concept                                              |
|----------------------|----------------------------------------------------------|----------------------------------------------------------------------|
| Cloud assistants     | Very strong intelligence and broad tools                 | Cloud-first; provider-controlled runtime/data boundary               |
| Local LLM runners    | Private, open models, offline                            | Often primarily chat/model playgrounds                               |
| Local agent/MCP apps | Tools and integrations                                   | May lack coherent personal Knowledge/Memory/Skills model             |
| Proposed product     | Private + Knowledge + Memory + Skills + MCP + multimodal | Must prove UX, local-model reliability and long-term retention value |

## 16. Closed decisions

CLOSED - iOS-first product.

CLOSED - Downloadable/local model inference is fundamental.

CLOSED - Local-first privacy: conversations, Knowledge, embeddings and retrieval stay on-device by default.

CLOSED - Multimodality is core, not a later add-on.

CLOSED - Knowledge/RAG is a first-class product concept.

CLOSED - Skills are supported and are behavior/instructions, not executable privilege.

CLOSED - Tools are capabilities and come from registered native, MCP or local retrieval executors.

CLOSED - Unified ToolRegistry hides executor type from the model.

CLOSED - Imported Skills can request but cannot auto-grant capabilities.

CLOSED - Progressive Skill loading to keep context small.

CLOSED - No JavaScript Skills in MVP.

CLOSED - Small, useful native iOS tool set rather than broad device-control promise.

CLOSED - MCP is the main route to open-ended external integrations.

CLOSED - Remote Streamable HTTP MCP first.

CLOSED - Explicit external-call privacy/approval boundary.

CLOSED - Separate embedding model for Knowledge retrieval.

CLOSED - ContextAssembler owns token budgeting.

CLOSED - Chat history is source-of-truth; derived Memory is rebuildable.

CLOSED - SQLite/GRDB is preferred for chat/entity relational storage.

CLOSED - No graph database needed initially.

CLOSED - Community Skill import/use should be free.

CLOSED - Do not meter local inference by messages/tokens.

CLOSED - Market the outcome (private personal AI), not the infrastructure.

## 17. Open decisions and validation work

OPEN - Final product name. “Nook” is only a candidate and requires trademark/App Store/domain validation.

OPEN - Exact minimum supported iPhone/iPad and RAM tiers.

OPEN - Exact flagship VLM/LLM after on-device agent/RAG/tool benchmarks.

OPEN - MLX-only long term vs adding llama.cpp/GGUF advanced support.

OPEN - Exact embedding model and embedding dimension.

OPEN - Vector layer: ObjectBox HNSW vs brute-force/other local index.

OPEN - Whether hybrid lexical + semantic retrieval ships in v1 or immediately after.

OPEN - Dedicated reranker in v1 (current leaning: no).

OPEN - Multimodal PDF-page RAG in v1 (current leaning: post-MVP).

OPEN - Full structured person/place/topic extraction in v1 (current leaning: later).

OPEN - Exact Skill package/manifest schema and portability guarantees.

OPEN - Exact Skill permission model and required-vs-optional capability semantics.

OPEN - Exact built-in Skill set.

OPEN - Exact Free/Pro limits and pricing.

OPEN - Cloud sync/backup strategy and whether it becomes a separate subscription.

OPEN - Model licensing/distribution and CDN strategy.

OPEN - DOCX/EPUB/webpage ingestion scope.

OPEN - Whether advanced model import is Pro-only or partially free.

OPEN - Whether Memory itself is entirely Pro or has a limited free trial/tier.

## 18. Recommended implementation sequence

### Phase 1 - Prove local intelligence

- 14. SwiftUI chat shell.
- 15. MLX model lifecycle/download manager.
- 16. One strong text model and one VLM.
- 17. Streaming, cancellation, memory-pressure handling, thermal/performance instrumentation.

### Phase 2 - Knowledge

- 18. Document import/extraction.
- 19. Chunking.
- 20. Embedding model.
- 21. Local vector search.
- 22. FTS/lexical retrieval if practical.
- 23. ContextAssembler.
- 24. Citations and Knowledge collection UI.

### Phase 3 - Agent/tool architecture

- 25. AgentSession loop.
- 26. AgentTool protocol.
- 27. ToolRegistry.
- 28. Structured tool-call parsing.
- 29. Permissions/confirmation model.
- 30. 2-3 native tools.

### Phase 4 - Skills

- 31. SkillManager.
- 32. SKILL.md parser/import.
- 33. Progressive discovery/activation.
- 34. Per-Skill capability and Knowledge grants.
- 35. Built-in Skills and personal Skill editor.

### Phase 5 - MCP

- 36. Swift MCP client.
- 37. Streamable HTTP server configuration.
- 38. Tool discovery/schema adaptation.
- 39. Approval UX.
- 40. Execution through existing ToolRegistry.

### Phase 6 - Memory and product polish

- 41. Cross-chat semantic + lexical retrieval.
- 42. Conversation summaries.
- 43. Explicit previous-conversation attachment.
- 44. Share Extension.

45. Model/Knowledge/Skill/Connection UX polish.
46. Later: entity extraction, aliases, relations and memory review/forget UI.

## 19. Architectural invariants / design tests

Use these as guardrails during implementation:

- A model can be replaced without rewriting Skills.
- A Skill can be installed without gaining executable privileges.
- A new MCP server can add capabilities without an app release.
- Knowledge can grow without model context growing proportionally.
- Long chat history can grow without every inference replaying it.
- Every derived Memory can point back to original evidence.
- External calls are distinguishable from local execution.
- Users can remove/rebuild derived indexes without deleting raw conversations.
- Normal users do not need to understand model formats, embeddings or MCP schemas to benefit.

### North-star user experience

The app should become more useful over time because it knows the user-selected Knowledge, can retrieve relevant prior context, and can acquire new workflows and connections - while retaining a clear, inspectable privacy boundary.

A representative future interaction: “What was that database Sarah recommended when we had dinner in London? Find its docs, compare it with the database we considered for Project Atlas, and save the evaluation criteria as a reusable Skill.” The system resolves prior Memory locally, uses external tools only with permission, combines Knowledge, and creates behavior without granting new executable code.

## 20. Immediate next steps

47. Build a small MLX Swift spike on 2-3 target iPhones and benchmark Gemma 3 4B, Qwen-family 3-4B VLMs and a low-memory ~1-2B baseline.
48. Define the AgentTool protocol and ToolRegistry before implementing Skills or MCP.
49. Prototype ContextAssembler with explicit token accounting.
50. Build Knowledge v0: PDF/Markdown → chunks → embeddings → local retrieval → citations.
51. Benchmark brute-force vector search vs ObjectBox on 1K, 10K, 50K and 100K chunks.
52. Draft Skill package v0.1: SKILL.md + optional manifest + permission declarations.
53. Prototype one end-to-end Skill using Knowledge + calendar native tool.
54. Add one no-risk MCP integration and validate the complete tool loop.
55. Prototype cross-chat Memory retrieval before building structured entity extraction.
56. Run competitive UX teardowns focused on onboarding, Knowledge, Skills/MCP configuration and monetization - not just feature checklists.

### Success criterion for the first prototype

On one iPhone, a user can download a local multimodal model, add project documents, ask a grounded question with citations, attach a screenshot for comparison, and invoke one approved external MCP tool - with stable memory use and a context window that remains bounded.